CO4-AT1-
Coding challenge

Name: Ajay Kumar Reddy

Reg.No:192425274

Course: CSA0712 - COMPUTER NETWORKS

Faculty: Dr. S. SALINI

Question 1 — TCP Client–Server Application — 50 Marks

Develop a using Python/C/C++.
The server should:
Create a TCP socket and bind it to a specified IP address and port.
Listen for incoming client connections.
Accept a client connection.
Receive a message from the client.
Process the message and send an appropriate response.
Continue serving multiple messages until the client sends "exit".
The client should:
Establish a TCP connection with the server.
Accept messages from the user.
Send the messages to the server.
Display the server's response.
Terminate the connection when "exit" is entered.

Answer:

Language Used: Python

tcp_server Code:

```
import socket

HOST = "127.0.0.1"

PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

server.bind((HOST, PORT))

server.listen(1)

print("================================")

print("          TCP SERVER")

print("================================")

print("Server started")

print("IP Address :", HOST)

print("Port       :", PORT)

print("Waiting for client...")

print()

connection, address = server.accept()

print("Client connected:", address)

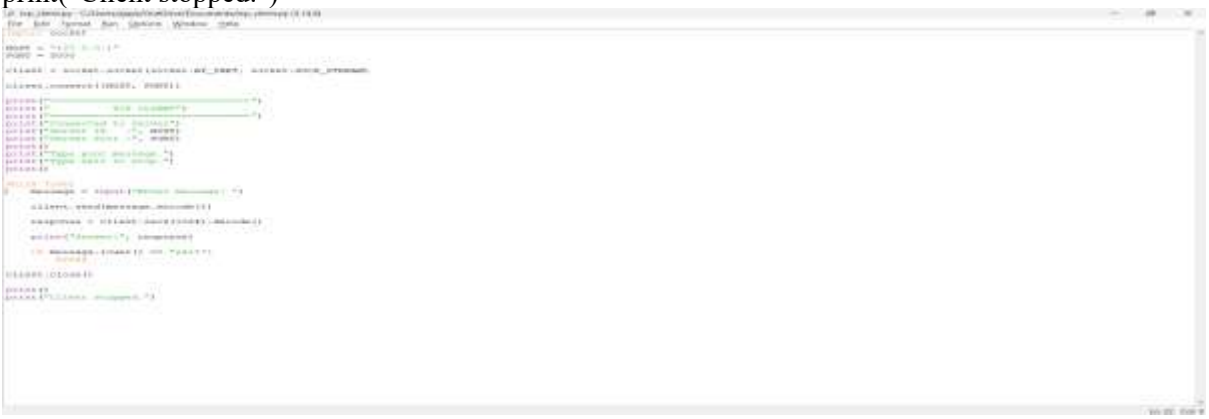
print()
```

```
while True:
    message = connection.recv(1024).decode()
    print("Client:", message)
    if message.lower() == "exit":
        connection.send(
            "Server received EXIT. Connection closed.".encode()
        )
        break
    response = "Server received: " + message
    connection.send(response.encode())
connection.close()
server.close()
print()
print("Server stopped.")
```



tcp_client Code:

```
import socket
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("================================")
print("          TCP CLIENT")
print("================================")
print("Connected to server")
print("Server IP   :", HOST)
print("Server Port :", PORT)
print()
print("Type your message.")
print("Type exit to stop.")
print()
while True:
    message = input("Enter message: ")
    client.send(message.encode())
    response = client.recv(1024).decode()
    print("Server:", response)
    if message.lower() == "exit":
        break
client.close()
print()
print("Client stopped.")
```



tcp_server Output:



tcp_client Output:



Question 2 — Reliable Data Transfer over UDP — 50 Marks

Develop a that demonstrates how reliability can be implemented at the transport layer.

The sender should:
Divide a message/file into packets.
Assign a sequence number to each packet.
Send packets to the receiver using UDP.
Wait for an acknowledgement (ACK).
Retransmit a packet if its ACK is not received within a specified timeout.
Display the sequence number of transmitted and retransmitted packets.

The receiver should:
Receive UDP packets.
Check the sequence number.
Send an ACK for correctly received packets.
Detect missing or duplicate packets.
Reassemble the original message in the correct order.

Answer:

Language Used: Python

udp_receiver Code:

```
import socket
HOST = "127.0.0.1"
PORT = 6000
receiver = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
receiver.bind((HOST, PORT))
print("========================================")
print("          UDP RECEIVER")
print("========================================")
print("Receiver started")
print("IP Address :", HOST)
print("Port       :", PORT)
print("Waiting for packets...")
print("----------------------------------------")
expected = 0
received_message = []
```

```
while True:
    data, address = receiver.recvfrom(1024)
    packet = data.decode()
    if packet == "END":
        break
    parts = packet.split("|", 2)
    sequence_number = int(parts[0])
    total_packets = int(parts[1])
    message_part = parts[2]
    print("Received Packet :", sequence_number)
    print("Data            :", message_part)
    if sequence_number == expected:
        print("Status          : Correct packet")
        received_message.append(message_part)
        ack = "ACK|" + str(sequence_number)
        receiver.sendto(
            ack.encode(),
            address
        )
        expected = expected + 1
    elif sequence_number < expected:
        print("Status          : Duplicate packet")
        ack = "ACK|" + str(sequence_number)
        receiver.sendto(
            ack.encode(),
            address
        )
    else:
        print("Status          : Missing packet detected")
        print("Expected Packet :", expected)
        ack = "ACK|" + str(expected - 1)
        receiver.sendto(
            ack.encode(),
            address
        )
    print("----------------------------------------")
```

```
print()
print("========================================")
print("      TRANSMISSION COMPLETE")
print("========================================")
print("Reassembled Message:")
print("".join(received_message))
print("----------------------------------------")
receiver.close()
print("UDP Receiver stopped.")
```

udp_receiver.py - C:/Users/ajayk/OneDrive/Documents/udp_receiver.py (3.14.6)

File Edit Format Run Options Window Help

```
import socket

HOST = "127.0.0.1"
PORT = 6000

receiver = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

receiver.bind((HOST, PORT))

print("========================================")
print("          UDP RECEIVER")
print("========================================")
print("Receiver started")
print("IP Address :", HOST)
print("Port       :", PORT)
print("Waiting for packets...")
print("----------------------------------------")

expected = 0
received_message = []

while True:

    data, address = receiver.recvfrom(1024)

    packet = data.decode()

    if packet == "END":
        break

    parts = packet.split("|", 2)

    sequence_number = int(parts[0])
    total_packets = int(parts[1])
    message_part = parts[2]

    print("Received Packet :", sequence_number)
    print("Data            :", message_part)

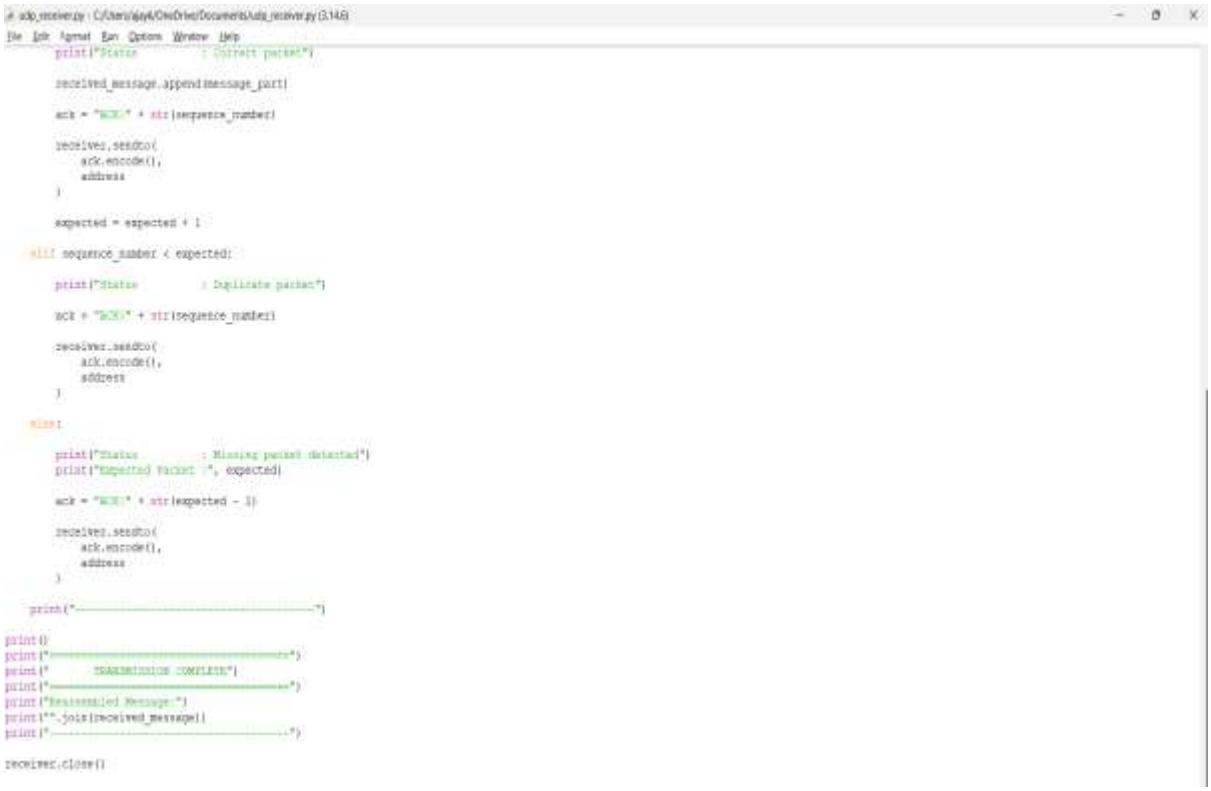
    if sequence_number == expected:

        print("Status          : Correct packet")

        received_message.append(message_part)

        ack = "ACK|" + str(sequence_number)

        receiver.sendto(
```



```
        print("Status          : Correct packet")

        received_message.append(message_part)

        ack = "ACK:" + str(sequence_number)

        receiver.sendto(
            ack.encode(),
            address
        )

        expected = expected + 1

    elif sequence_number < expected:

        print("Status          : Duplicate packet")

        ack = "ACK:" + str(sequence_number)

        receiver.sendto(
            ack.encode(),
            address
        )

    else:

        print("Status          : Missing packet detected")
        print("Expected Packet :", expected)

        ack = "ACK:" + str(expected - 1)

        receiver.sendto(
            ack.encode(),
            address
        )

    print("----------------------------------------")

print()
print("========================================")
print("       TRANSMISSION COMPLETE")
print("========================================")
print("Reassembled Message:")
print("".join(received_message))
print("----------------------------------------")

receiver.close()
```

udp_sender Code:

```
import socket

HOST = "127.0.0.1"

PORT = 6000

PACKET_SIZE = 5

TIMEOUT = 2

sender = socket.socket(

    socket.AF_INET,

    socket.SOCK_DGRAM

)

sender.settimeout(TIMEOUT)

print("========================================")

print("           UDP SENDER")

print("========================================")

message = input("Enter message: ")

packets = []

for i in range(0, len(message), PACKET_SIZE):

    part = message[i:i + PACKET_SIZE]

    packets.append(part)

print()
```

```
print("Original Message :", message)
print("Packet Size      :", PACKET_SIZE)
print("Total Packets    :", len(packets))
print("----------------------------------------")
loss_packet = int(
    input("Enter packet number to simulate loss (0 for none): ")
)
print("----------------------------------------")
for sequence_number in range(len(packets)):
    message_part = packets[sequence_number]
    packet = (
        str(sequence_number)
        + "|"
        + str(len(packets))
        + "|"
        + message_part
    )
    acknowledged = False
    first_attempt = True
    while acknowledged == False:
        if sequence_number == loss_packet and first_attempt:
            print(
                "Packet",
                sequence_number,
                "SIMULATED LOSS!"
            )
            first_attempt = False
        else:
            if first_attempt:
                print(
                    "Sending Packet :",
                    sequence_number,
                    "Data :",
                    message_part
                )
            else:
```

```
                print(
                    "RETRANSMITTING Packet :",
                    sequence_number,
                    "Data :",
                    message_part
                )
            sender.sendto(
                packet.encode(),
                (HOST, PORT)
            )
        try:
            data, address = sender.recvfrom(1024)
            ack = data.decode()
            ack_number = int(
                ack.split("|")[1]
            )
            print(
                "ACK received for Packet :",
                ack_number
            )
            if ack_number == sequence_number:
                acknowledged = True
        except socket.timeout:
            print(
                "TIMEOUT! ACK not received for Packet :",
                sequence_number
            )
            first_attempt = False
    print("---------------------------------------")
sender.sendto(
    "END".encode(),
    (HOST, PORT)
)
print()
print("=======================================")
print("       TRANSMISSION COMPLETE")
```

print("==")

print("All packets sent successfully.")

print("END signal sent.")

print("UDP Sender stopped.")

sender.close()



```
udp_sender.py - C:/Users/gayat/OneDrive/Documents/udp_sender.py (3.14.0)
File Edit Format Run Options Window Help
import socket

HOST = "127.0.0.1"
PORT = 6000

PACKET_SIZE = 5
TIMEOUT = 2

sender = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)

sender.settimeout(TIMEOUT)

print("========================================")
print("          UDP SENDER")
print("========================================")

message = input("Enter message: ")

packets = []

for i in range(0, len(message), PACKET_SIZE):

    part = message[i:i + PACKET_SIZE]

    packets.append(part)

print()
print("Original Message :", message)
print("Packet Size      :", PACKET_SIZE)
print("Total Packets    :", len(packets))

print("----------------------------------------")

loss_packet = int(
    input("Enter packet number to simulate loss (0 for none): ")
)

print("----------------------------------------")

for sequence_number in range(len(packets)):

    message_part = packets[sequence_number]

    packet = (
        str(sequence_number)
```



```
udp_sender.py - C:/Users/gayat/OneDrive/Documents/udp_sender.py (3.14.0)
File Edit Format Run Options Window Help
        + ":"
        + message_part
    )

    acknowledged = False
    first_attempt = True

    while acknowledged == False:

        if sequence_number == loss_packet and first_attempt:

            print(
                "Packet",
                sequence_number,
                "SIMULATED LOSS!"
            )

            first_attempt = False

        else:

            if first_attempt:

                print(
                    "Sending Packet :",
                    sequence_number,
                    "Data :",
                    message_part
                )

            else:

                print(
                    "RETRANSMITTING Packet :",
                    sequence_number,
                    "Data :",
                    message_part
                )

            sender.sendto(
                packet.encode(),
                (HOST, PORT)
            )

        try:

            data, address = sender.recvfrom(1024)
```

```
udp_sender.py - C:/Users/pavik/OneDrive/Documents/udp_sender.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
            sender.sendto(
                packet.encode(),
                (HOST, PORT)
            )

        try:

            data, address = sender.recvfrom(1024)

            ack = data.decode()

            ack_number = int(
                ack.split(":")[1]
            )

            print(
                "ACK received for packet :",
                ack_number
            )

            if ack_number == sequence_number:

                acknowledged = True

        except socket.timeout:

            print(
                "TIMEOUT: ACK not received for packet :",
                sequence_number
            )

            first_attempt = False

    print("-----------------------------------------")

sender.sendto(
    "END".encode(),
    (HOST, PORT)
)

print()
print("=========================================")
print("       TRANSMISSION COMPLETE")
print("=========================================")
print("All packets sent successfully.")
print("END signal sent.")
print("UDP Sender stopped.")
```

udp_receiver Output:



udp_sender Output:

